

The logo for CYBERCLASS, featuring the word "CYBERCLASS" in a bold, blue, sans-serif font with a slight 3D effect, set against a light gray rectangular background.

TECNICO EN CIBERSEGURIDAD

Participante: Luis Mario Reynoso

Técnico: Ciberseguridad

Nombre del Módulo: RIPTOGRAFÍA Y CIFRADO DE DATOS;

Facilitador: Kevin Feliz Henríquez

Centro: Cyberclass

Nombre Actividad: Laboratorio: Repaso de Fundamentos de Programación con Python

Descripción: En esta actividad estaremos creando un programa, con el lenguaje Python que es Transición Biológica con Python, que permite determinar la etapa biológica correspondiente de acuerdo a la edad.

¿Qué construiremos?

Desarrollaremos una aplicación que permita:

 Introducir una edad.

 Determinar la etapa biológica correspondiente.

 Guardar consultas realizadas.

 Visualizar un historial de resultados.

 Salir del programa mediante un menú interactivo.

FASE 1: Preparación del Entorno

 Paso 1: Instalar Python

¿Qué es Python?

Python es un lenguaje de programación sencillo, poderoso y ampliamente utilizado en:

 Ciberseguridad

 Desarrollo Web

 Inteligencia Artificial

 Ciencia de Datos

 Automatización

Descarga

Visitar: <https://www.python.org>

Paso 2: Instalar Visual Studio Code

¿Qué es VS Code?

Es un editor de código que nos permitirá escribir y ejecutar programas.

Descargar desde:

 <https://code.visualstudio.com>



Paso 3: Instalar la Extensión de Python

Abrir VS Code.

Seleccionar:



Extensions

Buscar: Python

Instalar la extensión publicada por Microsoft.



FASE 2: Creación del Proyecto

Abrir CMD

Presionar:

⌘ Windows + R

Escribir:

cmd

Ir al escritorio

cd Desktop

Crear carpeta del proyecto

mkdir proyecto_biologico

Entrar a la carpeta

cd proyecto_biologico

Abrir VS Code desde la terminal

code .

FASE 3: Comprendiendo el Programa

Variable global: **historial**

```
historial = []
```

Esta lista se utiliza para guardar en memoria los resultados generados por el programa.

Cada vez que el usuario introduce una edad y el sistema la clasifica, se guarda un registro dentro de esta lista.

Ese registro contiene:

- La edad digitada.
- El resultado o clasificación de esa edad.

```
1 #Lista para guardar los resultados en memoria
2 historial = []
3
```

Ejemplo de un registro guardado:

```
{
    'edad': 18,
    'resultado': 'Eres un/a adulto/a en progreso'
}
```

Función **solicitar_edad()**

```
def solicitar_edad():
```

Esta función se encarga de pedirle al usuario que digite su edad.

Dentro de esta función se utiliza un ciclo while True, que permite repetir la solicitud hasta que el usuario escriba un valor válido.

```
entrada = input('Favor digite su edad: ')
```

Aquí se captura la edad escrita por el usuario.

```
if entrada.lstrip('-').isdigit():
```

Esta condición verifica si el valor digitado es numérico.

El método lstrip('-') permite aceptar números negativos, por ejemplo -1.

Esto es útil porque el programa tiene una condición para edades menores que cero.

```
return int(entrada)
```

Si el valor es correcto, se convierte a número entero y se retorna.

else:

```
    print('Error al digitar su edad, favor digitar un valor numerico.')
```

Si el usuario escribe letras, símbolos o un dato inválido, se muestra un mensaje de error y el programa vuelve a pedir la edad.

```
def solicitar_edad():
    while True:
        entrada = input('Favor de digite su edad:')
        #Permite Números positivos, cero y negativos
        if entrada.lstrip('-').isdigit():
            return int(entrada)
        else:
            print('Error al digitar a su edad, favor digitar un valor numerico.')
```

Función `clasificar_edad(edad)`

`def clasificar_edad(edad):`

Esta función recibe como parámetro la edad digitada por el usuario y se encarga de determinar a qué etapa biológica pertenece.

Dentro de la función se crea una lista llamada `etapas`.

```
etapas = [  
    (lambda e: e < 0, 'Aun no has nacido'),  
    (lambda e: e == 0, 'Eres un/a neonato/a'),  
    ...  
]
```

Cada elemento de la lista contiene dos partes:

1. Una condición.
2. El mensaje que se debe mostrar si esa condición se cumple.

Por ejemplo:

```
(lambda e: e >= 18 and e <= 29, 'Eres un/a adulto/a en progreso')
```

Esta línea significa que si la edad está entre 18 y 29 años, el resultado será:

Eres un/a adulto/a en progreso

Luego se utiliza un ciclo `for`:

`for condicion, mensaje in etapas:`

Este ciclo recorre todas las condiciones de la lista.

`if condicion(edad):`

`return mensaje`

Si la condición se cumple, la función devuelve el mensaje correspondiente.

Esta función ayuda a evitar tener muchos `if`, `elif` directamente en el programa principal, haciendo que el código sea más limpio y organizado.

```
def clasificar_edad(edad):  
    etapas = [  
        (lambda e: e < 0, 'Aun no has nacido'),  
        (lambda e: e == 0, 'Eres un/a neonato/a'),  
        (lambda e: e >= 1 and e <=13, 'Eres un/a niño/a'),  
        (lambda e: e >= 14 and e <=17, 'Eres un/a joven'),  
        (lambda e: e >= 18 and e <=29, 'Eres un/a adulto/a en progreso'),  
        (lambda e: e >= 30 and e <=59, 'Eres un/a adulto/a realido/a'),  
        (lambda e: e >= 60 and e <=79, 'Eres un/a evejeciente'),  
        (lambda e: e >= 80 and e <=99, 'Eres un/a Geriatico/a'),  
        (lambda e: e >= 100 and e <=120, 'Eres un/a milagro '),  
        (lambda e: e >= 121 and e <=139, 'Eres un/a leyenda (record guiness)'),  
        (lambda e: e >= 140, 'jabaldor/a')  
    ]  
  
    #Uso de for para recorrer las condiciones  
    for condicion,mensaje in etapas:  
        if condicion(edad):  
            return mensaje
```

Función `guardar_resultado(edad, resultado)`

```
def guardar_resultado(edad, resultado):
```

Esta función se encarga de guardar en memoria la edad consultada y su resultado.

```
    historial.append({  
        'edad': edad,  
        'resultado': resultado  
    })
```

```
def guardar_resultado(edad, resultado):  
    historial.append({  
        'edad': edad,  
        'resultado': resultado  
    })
```

Aquí se agrega un nuevo registro a la lista historial.

El método `append()` sirve para añadir un nuevo elemento al final de una lista.

Cada resultado queda guardado mientras el programa siga abierto.

Función `mostrar_historial()`

```
def mostrar_historial():
```

Esta función muestra todos los resultados guardados en memoria.

Primero verifica si el historial está vacío:

```
if len(historial) == 0:  
    print('No hay resultados guardados todavía.')
```

Si no hay consultas realizadas, muestra un mensaje indicando que todavía no existen resultados.

Si sí hay resultados, se ejecuta el bloque `else`:

```
else:  
    print("\nHistorial de resultados:')
```

Luego se utiliza un ciclo `for` para recorrer cada registro guardado:

```
for indice, registro in enumerate(historial, start=1):
```

La función `enumerate()` permite recorrer la lista y al mismo tiempo generar un número de orden para cada resultado.

El parámetro `start=1` hace que el conteo empiece desde 1 en vez de 0.

```
    print(f'{indice}. Edad: {registro["edad"]} - Resultado: {registro["resultado"]}')
```

Esta línea imprime cada resultado del historial con su número, edad y clasificación.

```
def mostrar_historial():  
    if len(historial) == 0:  
        print('No hay resultados guardados todaia.')    else:  
        print("\nHistorial de resultados:')  
        #Uso de for para mostrar los resultados almacenados en memoria  
        for indice, registro in enumerate(historial, start=1):  
            print (f'{indice}, Edad: {registro["edad"]} - Resultado: {registro["resultado"]}')
```

Función `transicion_biologica()`

`def transicion_biologica():`

Esta función representa el proceso principal de clasificación de edad.

Primero muestra el título del programa:

```
print('\nIdentificador de Transicion Biologica')
```

Luego solicita la edad llamando a la función `solicitar_edad()`:

```
edad = solicitar_edad()
```

Después clasifica la edad usando la función `clasificar_edad()`:

```
resultado = clasificar_edad(edad)
```

Luego imprime el resultado:

```
print(resultado)
```

Finalmente, guarda el resultado en el historial:

```
guardar_resultado(edad, resultado)
```

Esta función conecta varias partes del programa: pedir edad, clasificarla, mostrar el resultado y guardarlo.

```
def transicion_biologica():  
    print('\nIdentificador de transicion Biologica')  
  
    edad = solicitar_edad()  
    resultado = clasificar_edad(edad)  
  
    print(resultado)  
    guardar_resultado(edad, resultado)
```

Función `menu()`

`def menu():`

Esta función contiene el menú principal de la aplicación.

Dentro de ella se utiliza un ciclo `while True` para que el menú se repita hasta que el usuario decida salir.

```
print('\n===== MENU PRINCIPAL =====')
```

```
print('1. Identificar transicion biologica')
```

```
print('2. Ver historial de resultados')
```

```
print('3. Salir')
```

Estas líneas muestran las opciones disponibles para el usuario.

```
opcion = input('Seleccione una opcion: ')
```

Aquí el usuario escribe la opción que desea ejecutar.

```
def menu():
    while True:
        print('\n===== Menu Principal =====')
        print ('1. Identificar transicion biologica')
        print('2. Ver hsitorial de resultados')
        print('3. Salir')

        opcion = input('Seleccione una opcion:')

        if opcion == '1':
            transicion_biologica()
        elif opcion == '2':
            mostrar_historial()
        elif opcion == '3':
            print ('Muchas gracias por usar nuetra aplicacion!!')
        else:
            print ('Opcion invalida. Favor seleccionar una opcion correcta.')

menu()
```

programa en ejecucion



```
C:\Windows\py.exe

===== Menu Principal =====
1. Identificar transicion biologica
2. Ver hsitorial de resultados
3. Salir
Seleccione una opcion:1_
```



```
C:\Windows\py.exe

===== Menu Principal =====
1. Identificar transicion biologica
2. Ver hsitorial de resultados
3. Salir
Seleccione una opcion:1

Identificador de transicion Biologica
Favor de digite su edad:20
Eres un/a adurto/a en progreso

===== Menu Principal =====
1. Identificar transicion biologica
2. Ver hsitorial de resultados
3. Salir
Seleccione una opcion:
```

